BarberSpringfield – Sistema de Gestión de Barbería

Plan de Migración

Versión: 1

Fecha: 15/10/2026

Control de Cambios

Versión	Causa del Cambio	Responsable del Cambio	Fecha del Cambio
1	Versión inicial	Samuel Tellez	15/10/2026

Tabla de contenido

Introducción 4

Objetivo 4

Alcance 4

Razones para la Migración de Datos 4

Base de Datos Origen y Posible Base de Datos Destino 5

Tipo de Migración 6

Metodología de la Migración de Datos 6

Planeación 6

Migración 9

Validación de datos 9

Técnica de Migración de Datos 10

Extracción 10

Diccionario de Datos de la Base de Datos Origen 10

Mapeo de Campos 12

Transformación 13

Carga 14

Validación 14

Estrategia de Migración de Datos 16

Referencias 17

Introducción

Objetivo

El presente Plan de Migración define, de forma ordenada y verificable, cómo se trasladan los datos del sistema de gestión de la barbería BarberSpringfield desde su base de datos de origen (SQLite, archivo db.sqlite3) hacia la base de datos de destino (MariaDB 10.4.32, esquema barberspringfield_db), sin pérdida ni alteración de la información de citas, clientes, barberos, servicios, inventario y notificaciones. El documento establece las fases, técnicas, validaciones y el plan de contingencia de la migración.

Alcance

Este documento está dirigido al equipo de desarrollo y administración del sistema (responsables técnicos de la migración), al propietario de la barbería como aprobador del plan y a quien brinde el soporte posterior. Cubre las tablas de las aplicaciones del proyecto Django (administracion, barberos, citas, clientes, inventario, notificaciones, usuarios y googlecalendar) y las tablas de autenticación; no modifica el código fuente, salvo la configuración de conexión.

Razones para la Migración de Datos

El sistema se desarrolló sobre SQLite, un motor embebido que guarda toda la base en un único archivo (db.sqlite3, 344 KB). Es práctico para desarrollo, pero limitado para operar una barbería real. La configuración del proyecto (settings.py) ya apunta a MariaDB, por lo que la migración consiste en poner los datos existentes en el motor definitivo. Las razones son:

Concurrencia: SQLite permite una sola escritura a la vez; MariaDB atiende varios usuarios simultáneos (clientes agendando, barberos y administrador trabajando a la vez).

Seguridad y acceso: un archivo local no tiene usuarios ni permisos; MariaDB gestiona cuentas, privilegios y acceso por red.

Integridad de datos: SQLite usa tipado flexible, es decir, no obliga a respetar el tamaño de un varchar ni el tipo declarado (SQLite, s. f.); MariaDB sí valida tipos, longitudes, claves foráneas y restricciones CHECK (MariaDB Foundation, s. f.).

Respaldo y herramientas: MariaDB permite respaldos con mysqldump o phpMyAdmin, administración gráfica y recuperación ante fallos.

Un único origen de verdad: el repositorio contiene un db.sqlite3 y tres volcados SQL de MariaDB; la migración deja una sola base oficial y un respaldo versionado.

Base de Datos Origen y Posible Base de Datos Destino

Base de datos origen: SQLite (archivo db.sqlite3). Contiene 23 tablas (17 de aplicación y autenticación, más tablas de sistema de Django) y unas 300 filas de negocio; las mayores son notificaciones_notificacion (166) y citas_cita (82).

Base de datos destino: MariaDB 10.4.32, motor InnoDB, juego de caracteres utf8mb4 con colación utf8mb4_unicode_ci, base barberspringfield_db (volcado de phpMyAdmin del 15/10/2026). Se compararon las siguientes alternativas (Tabla 1).

Tabla 1

Alternativas de base de datos destino

Alternativa	Ventajas	Desventajas
MariaDB 10.4 (elegida)	Ya configurada en settings.py y disponible en el entorno (XAMPP/phpMyAdmin); multiusuario; validación de tipos y claves foráneas; soporte de Django (Django Software Foundation, s. f.-a).	Colación no sensible a mayúsculas (puede chocar con valores únicos); JSON se guarda como texto con validación; requiere administrar el servidor.
PostgreSQL	Tipos estrictos y jsonb nativo; muy robusto para crecimiento.	Exige cambiar de controlador y reconfigurar el entorno; no es el motor ya preparado en el proyecto.
Mantener SQLite	Sin administración; ya existe el archivo.	Una sola escritura simultánea; sin usuarios ni acceso en red; no valida longitudes; no apto para producción.

Nota. Elaboración propia a partir de la configuración del proyecto.

Tipo de Migración

La migración es heterogénea: se pasa de un motor embebido (SQLite) a uno cliente-servidor distinto (MariaDB), con diferentes sistemas de tipos, manejo de claves autoincrementales y colaciones. Como el acceso a datos se hace con el ORM de Django, la diferencia se resuelve recreando el esquema con las migraciones del framework y transportando los datos en un formato independiente del motor (JSON con dumpdata y loaddata) (Django Software Foundation, s. f.-b).

Metodología de la Migración de Datos

Planeación

La planeación define qué se migra, cómo, cuándo y con qué criterios se da por exitosa. A continuación se explica cada componente de la Tabla 2, aplicado a BarberSpringfield.

Tabla 2

Componentes de la planeación de la migración

Componente	Actividades
Planeación	Alcance, estrategias y fases Identificar requerimientos Análisis de riesgos Análisis de la situación actual y final Determinar la factibilidad técnica y económica
Aprobación del plan	Revisión del plan con los involucrados Obtención del visto bueno del plan
Análisis del esquema actual y del nuevo esquema	Análisis de integridad Análisis de consistencia Análisis de valores nulos y tipos de dato Verificar el tamaño de los objetos y de la base de datos Verificar las características de la base de datos

Componente	Actividades
Selección y/o desarrollo de la aplicación para migrar	Evaluación de las alternativas Compra o desarrollo de la aplicación
Pruebas	Pruebas del proceso de migración Pruebas de las aplicaciones que usarán la base de datos migrada

Alcance, estrategias y fases. Se migran todas las tablas de aplicación y las de usuarios y grupos. No se migran django_session, django_admin_log, django_content_type ni auth_permission, porque Django los regenera con migrate. Fases: respaldo, creación del destino, extracción, carga, validación y cierre.

Requerimientos. Servidor MariaDB 10.4 activo; base barberspringfield_db con utf8mb4; usuario de aplicación; acceso al archivo db.sqlite3; copia de la carpeta media/ (imágenes); ventana de mantenimiento.

Análisis de riesgos. Los riesgos y su mitigación se resumen en la Tabla 3.

Tabla 3

Riesgos de la migración y su mitigación

Riesgo	Prob.	Impacto	Mitigación
Pérdida o corrupción de datos	Baja	Alto	Respaldo previo del archivo db.sqlite3; el origen no se modifica; validación de conteos.
Textos más largos que el varchar del destino (SQLite no valida longitud)	Media	Medio	Consulta previa con MAX(LENGTH()) por columna antes de cargar.
Pérdida de integridad referencial por orden de carga	Media	Alto	Cargar maestras antes que dependientes; PRAGMA foreign_key_check en el origen.
Conversión de tipos (bool, fechas en texto, JSON, decimales)	Media	Medio	Mapeo de campos y prueba con los 82 registros de citas_cita.
Colación no sensible a mayúsculas en el destino (usuarios o correos repetidos)	Media	Medio	Revisar duplicados por LOWER() antes de migrar.
Exposición de credenciales (tokens de Google, API keys)	Media	Alto	Credenciales en .env; no versionar respaldos con datos sensibles.
Citas creadas en el origen durante la migración	Media	Medio	Ventana fuera del horario de atención y sistema en mantenimiento.

Análisis de la situación actual y final. Actual: archivo SQLite local, sin usuarios ni control de concurrencia. Final: MariaDB cliente-servidor con tipos validados, restricciones activas y respaldo propio.

Factibilidad técnica y económica. Viable: Django soporta ambos motores y el volumen es mínimo. El costo es bajo, pues MariaDB es de código abierto y ya está instalado; solo se invierten unas horas del equipo.

Aprobación del plan. Se revisa el documento con el propietario y el equipo técnico y se registra el visto bueno en el control de cambios antes de ejecutar cualquier paso.

Análisis del esquema actual y del nuevo. Integridad: comprobar que cada clave foránea tenga su registro padre (citas_cita con clientes_cliente, citas_servicio y barberos_barbero; clientes_cliente con auth_user). Consistencia: que stock_anterior y stock_nuevo cuadren con stock_actual y que cada estado de cita sea válido. Valores nulos y tipos: cedula y telefono de clientes_cliente admiten NULL; el resto de columnas son NOT NULL y SQLite puede contener cadenas vacías o tipos mezclados. Tamaño: 344 KB en origen, sin problema para el destino. Características: pasa de tipado flexible a tipado estricto, de INTEGER PRIMARY KEY a AUTO_INCREMENT y de fechas en texto a DATETIME(6).

Selección o desarrollo de la aplicación para migrar. Alternativas: (a) herramientas externas (por ejemplo, convertidores de SQLite a MySQL), (b) scripts SQL manuales, (c) las herramientas de Django: migrate, dumpdata y loaddata. Se elige (c) porque no tiene costo, respeta las reglas del ORM y es independiente del motor.

Pruebas. Del proceso: ensayo completo sobre una copia de prueba hasta que las validaciones coincidan. De la aplicación: inicio de sesión (cliente, barbero y administrador),

agendar y cancelar una cita, dashboard del barbero, cierre de caja y reportes PDF, movimientos de inventario, notificaciones y sincronización con Google Calendar.

Migración

La migración se ejecuta en una ventana de mantenimiento con el sistema sin usuarios. Cada paso se registra con fecha y hora en la memoria técnica. Las actividades se resumen en la Tabla 4.

Tabla 4

Actividades de la etapa de migración

Etapa	Actividad
Migración	Respaldo de la base de datos origen
Migración	Extracción, transformación y carga
Migración	Respaldo de la base de datos destino

Respaldo de la base de datos origen. Se copia el archivo db.sqlite3 (y la carpeta media/) y se genera un volcado de texto con sqlite3 db.sqlite3 .dump. Es el punto de retorno si algo falla.

Extracción, transformación y carga. Extracción con dumpdata desde SQLite; transformación de tipos mediante el ORM; carga con loaddata en MariaDB, tras crear el esquema con migrate. Los pasos están en la sección Técnica de Migración de Datos.

Respaldo de la base de datos destino. Terminada y validada la carga, se exporta barberspringfield_db (mysqldump o phpMyAdmin) como línea base del nuevo sistema antes de abrirlo a los usuarios.

Validación de datos

La validación confirma que los datos migrados son completos y correctos y que el sistema funciona sobre MariaDB. Las actividades se resumen en la Tabla 5.

Medición de resultados. Se comparan por tabla el número de registros y la suma de campos numéricos clave (precio, stock_actual, duracion_total). El criterio de éxito es cero diferencias en las tablas de aplicación.

Análisis de resultados. Cada diferencia se documenta, se corrige y se repite la validación; luego se ejecutan las pruebas funcionales. Con el volcado de destino entregado (barberspringfield_db.sql) los conteos de las tablas de aplicación coinciden con el origen (Tabla 8).

Cierre del proyecto. Reporte final con conteos e incidencias; resumen ejecutivo para el propietario; memoria técnica con los comandos ejecutados, los respaldos y las lecciones aprendidas.

Tabla 5

Actividades de la etapa de resultados

Etapa	Actividad
Resultados	Medición de resultados
Resultados	Análisis de resultados
Cierre del proyecto	Reporte final
Cierre del proyecto	Resumen ejecutivo
Cierre del proyecto	Memoria técnica

Técnica de Migración de Datos

Extracción

Diccionario de Datos de la Base de Datos Origen

Diccionario de datos resumido del origen (db.sqlite3). Todas las tablas tienen clave primaria id (Tabla 6).

Tabla 6

Diccionario de datos resumido de la base de datos origen

Tabla	Rol	Filas	Claves foráneas
auth_group	Maestra	1	—
auth_user	Maestra	9	—
administracion_sitio	Maestra	1	—
citas_servicio	Maestra	6	—
barberos_barbero	Maestra	4	—
inventario_producto	Maestra	4	—
citas_suscriptornewsletter	Maestra	0	—
auth_user_groups	Dependiente	3	user_id → auth_user; group_id → auth_group
usuarios_perfil	Dependiente	0	usuario_id → auth_user
clientes_cliente	Dependiente	8	user_id → auth_user
citas_cita	Dependiente	82	cliente_id → clientes_cliente; servicio_id → citas_servicio; barbero_id → barberos_barbero
inventario_movimientoinventario	Dependiente	16	producto_id → inventario_producto; usuario_id → auth_user
googlecalendar_cuentagoogle	Dependiente	0	usuario_id → auth_user
googlecalendar_citaeventogoogle	Dependiente	2	cita_id → citas_cita; usuario_id → auth_user
notificaciones_notificacion	Dependiente	166	cita_id → citas_cita; usuario_id → auth_user
notificaciones_suscripcionturno	Dependiente	0	barbero_id → barberos_barbero; usuario_id → auth_user

Nota. Tablas vacías en el origen: 0 filas. No se incluyen las tablas de sistema de Django (django_migrations, django_content_type, auth_permission, django_admin_log y django_session), que se regeneran. Elaboración propia a partir de db.sqlite3.

Tablas maestras y dependientes. Maestras (no dependen de otras): auth_group, auth_user, administracion_sitio, citas_servicio, barberos_barbero, inventario_producto y citas_suscriptornewsletter. Dependientes: las demás; la más crítica es citas_cita, que depende de tres maestras. Orden de carga: (1) auth_group, auth_user, auth_user_groups; (2) administracion_sitio, citas_servicio, barberos_barbero, inventario_producto,

citas_suscriptornewsletter; (3) usuarios_perfil, clientes_cliente; (4) citas_cita; (5) inventario_movimientoinventario, googlecalendar_* y notificaciones_*.

Mapeo de Campos

Mapeo de campos desde el origen (SQLite) hacia el destino (MariaDB 10.4), según el esquema del volcado barberspringfield_db.sql (Tabla 7).

Tabla 7

Mapeo de campos origen–destino

Campo(s)	**Tipo en SQLite**	**Tipo en MariaDB**	**Observación**
id (todas las tablas)	INTEGER PRIMARY KEY	bigint(20) / int(11) AUTO_INCREMENT	El contador se ajusta solo al cargar con ids explícitos
activo leida is_staff is_active correo_confirmado recordatorio_enviado historial_archivado	bool (0/1)	tinyint(1)	Directo
citas_cita.analisis_ia	TEXT	longtext utf8mb4_bin + CHECK json_valid	Verificar que todo el contenido sea JSON válido
numero_resenas duracion_total cortes_* recompensas_disponibles stock_* cantidad precio duracion	integer unsigned	int(10) UNSIGNED + CHECK ≥ 0	Verificar que no haya negativos
fecha_registro creada creado actualizado last_login date_joined	datetime (texto)	datetime(6)	Conversión de texto a fecha; revisar zona horaria

Campo(s)	Tipo en SQLite	Tipo en MariaDB	Observación
citas_cita.fecha notificaciones_suscripcionturno.fecha	date	date	Directo
barberos_barbero. jornada_inicio / jornada_fin	time	time(6)	Directo
calificacion; precio_unitario	decimal	decimal(2,1); decimal(10,2)	SQLite guarda como número flotante: revisar redondeo
descripcion mensaje hero_* nosotros_* testimonio_* token_*	text	longtext	Directo
citas_cita.hora (ej. 08:00 AM)	varchar(20)	varchar(20)	Se conserva como texto
varchar(n) en general	varchar(n) sin validación	varchar(n) con validación	Comprobar longitudes máximas antes de cargar
*_archivo *_url imagen	varchar	varchar(100) / varchar(200)	Copiar carpeta media/ al servidor

Nota. Elaboración propia con base en el esquema de origen, el volcado de destino, SQLite (s. f.) y MariaDB Foundation (s. f.).

Transformación

Transformación: paso a paso de la preparación de los datos y de la base de destino.

Paso 1. Detener el uso del sistema y respaldar el origen:

```
cp db.sqlite3 backup_pre_migracion.sqlite3
sqlite3 db.sqlite3 .dump > backup_pre_migracion.sql
```

Paso 2. Depurar el origen: integridad, huérfanos, duplicados por mayúsculas y longitudes máximas.

```
PRAGMA integrity_check;
```

```
PRAGMA foreign_key_check;
SELECT LOWER(username), COUNT(*) FROM auth_user GROUP BY LOWER(username)
HAVING COUNT(*)>1;
SELECT MAX(LENGTH(nombre)) FROM clientes_cliente;   -- comparar con
varchar(200)
```

Paso 3. Extraer los datos desde SQLite con la configuración del origen activa, excluyendo tablas que Django regenera (Django Software Foundation, s. f.-b):

```
python manage.py dumpdata --natural-foreign --natural-primary \
  --exclude contenttypes --exclude auth.permission \
  --exclude sessions --exclude admin.logentry \
  --indent 2 -o datos_migracion.json
```

Paso 4. Crear la base de destino en MariaDB:

```
CREATE DATABASE barberspringfield_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Paso 5. Cambiar settings.py al destino MariaDB (ya definido en el proyecto: motor mysql, host 127.0.0.1, puerto 3306, charset utf8mb4), idealmente con usuario y contraseña desde variables de entorno, y crear el esquema vacío:

```
python manage.py migrate
```

Carga

Importación de los datos al destino. loaddata carga todo en una sola transacción y respeta las claves foráneas; si algo falla, no queda nada a medias. Si existen señales (post_save) que generen notificaciones, deben revisarse para que no dupliquen datos durante la carga.

```
python manage.py loaddata datos_migracion.json

mysqldump -u root -p --single-transaction barberspringfield_db >
backup_post_migracion.sql
```

Después se copia la carpeta media/ al servidor de destino y se confirman las rutas de las imágenes.

Validación

Validación de registros. Consulta de conteo con variables en el destino (MariaDB, variable @) y su equivalente en el origen (SQLite); los resultados deben coincidir.

```
-- DESTINO (MariaDB)
SELECT COUNT(*) INTO @total_citas    FROM citas_cita;
SELECT COUNT(*) INTO @total_clientes FROM clientes_cliente;
SELECT COUNT(*) INTO @total_notif    FROM notificaciones_notificacion;
SELECT @total_citas, @total_clientes, @total_notif;   -- esperado: 82, 8,
166

-- Integridad referencial en el destino (debe devolver 0)
SELECT COUNT(*) FROM citas_cita c LEFT JOIN clientes_cliente k ON
k.id=c.cliente_id WHERE k.id IS NULL;

-- Consistencia del JSON (debe devolver 0)
SELECT COUNT(*) FROM citas_cita WHERE JSON_VALID(analisis_ia)=0;

-- ORIGEN (SQLite, sin variables)
SELECT COUNT(*) FROM citas_cita;   -- esperado: 82
```

La Tabla 8 resume la comparación realizada entre el origen (db.sqlite3) y el volcado de destino entregado.

Tabla 8

Comparación de registros entre origen y destino

Tabla	**SQLite (origen)**	**MariaDB (destino)**	**Coincide**
administracion_sitio	1	1	Sí
auth_group	1	1	Sí
auth_user	9	9	Sí
auth_user_groups	3	3	Sí
barberos_barbero	4	4	Sí
citas_servicio	6	6	Sí
clientes_cliente	8	8	Sí
citas_cita	82	82	Sí
inventario_producto	4	4	Sí
inventario_movimientoinventario	16	16	Sí
googlecalendar_citaeventogoogle	2	2	Sí
notificaciones_notificacion	166	166	Sí
Demás tablas de aplicación (6)	0	0	Sí
django_session / django_admin_log	60 / 34	60 / 34	Sí
auth_permission / django_content_type / django_migrations	80 / 20 / 78	76 / 19 / 77	No (se regeneran con migrate)

Estrategia de Migración de Datos

Estrategia elegida: Big Bang (migración única). Se migra toda la base en una sola ventana de mantenimiento y el sistema pasa de SQLite a MariaDB de una vez.

Justificación. (1) El volumen es muy pequeño (aprox. 300 filas de negocio; 344 KB), por lo que la carga completa toma minutos. (2) Es una sola sede con tráfico bajo y se puede programar fuera del horario de atención. (3) Una migración incremental exigiría mantener sincronizados dos motores a la vez, con riesgo de inconsistencias y sin necesidad real. (4) El riesgo es controlado porque el origen no se modifica y se puede volver a él en cualquier momento. Las fases se muestran en la Tabla 9.

Tabla 9

Fases y duración estimada de la migración

Fase	Actividad	Duración estimada
1	Aprobación del plan y ensayo en entorno de prueba	1 día
2	Aviso a usuarios y congelación del origen	15 min
3	Respaldo del origen y extracción (dumpdata)	10 min
4	Creación del destino, migrate y loaddata	15 min
5	Copia de media/ y respaldo del destino	10 min
6	Validación de conteos y pruebas funcionales	30 min
7	Apertura al público y monitoreo	1 día

Nota. Elaboración propia.

Plan de contingencia (rollback). Si la validación falla o aparece un error crítico, se restaura settings.py al origen SQLite (archivo intacto), se reabre el sistema y se analiza la causa antes de reintentar. El origen se conserva sin cambios al menos 30 días tras la migración.

Referencias

Django Software Foundation. (s. f.-a). *Databases* [Documentación de Django]. https://docs.djangoproject.com/en/stable/ref/databases/

Django Software Foundation. (s. f.-b). *django-admin and manage.py* [Documentación de Django]. https://docs.djangoproject.com/en/stable/ref/django-admin/

MariaDB Foundation. (s. f.). *Data types* [Base de conocimiento de MariaDB]. https://mariadb.com/kb/en/data-types/

SQLite. (s. f.). *Datatypes in SQLite* [Documentación de SQLite]. https://www.sqlite.org/datatype3.html